

CASE STUDY REPORT

TITLE

Enterprise Information System: Building a Complete
Open-Source Development Environment

NAME

RUSHABH TOLIA [92400103461]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026–2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS

EC-6

BATCH

B

CASE STUDY REPORT

Enterprise Information System: Building a Complete Open-Source Development Environment

TITLE :

Enterprise Information System: Building a Complete Open-Source Development Environment

Course Name : OPEN SOURCE TECHNOLOGIES(01CE0526)

COs Mapped : CO1,CO2,CO3,CO4,CO5

SDGs Mapped : SDG 4 , SDG 9 , SDG 17

Abstract

This case study presents a complete open-source development environment for a university Library Management System. The proposed environment covers version control, web application development, relational database management, API and browser testing, performance testing, security testing, deployment, and CI/CD automation. Git and GitHub manage source control and collaboration; Laravel provides the application layer; PostgreSQL stores transactional library data; Nginx and PHP-FPM provide the deployment layer; Postman, Selenium, Apache JMeter, and OWASP ZAP provide different testing capabilities; and GitHub Actions connects development, testing, and deployment into a repeatable workflow. The design emphasizes cost effectiveness, scalability, security, maintainability, and reproducibility.

Keywords—Open Source Technologies, Library Management System, Laravel, PostgreSQL, GitHub Actions, Nginx, Selenium, JMeter, OWASP ZAP, CI/CD

I. Introduction

A software company has received a university contract to develop a Library Management System that must be cost-effective, scalable, secure, and fully based on open-source software. The case therefore requires a complete development environment rather than a single application framework. The selected toolchain connects source control, application development, data management, testing, security, deployment, and maintenance.

The proposed environment is important because the library system contains structured and transactional information such as books, members, loans, returns, fines, and related reports. The case study aims to design a reproducible workflow in which each tool has a defined role and can be connected through GitHub Actions. The scope covers development through maintenance, including automated testing and deployment.

II. Organization/Application Background

The application is a Library Management System for a university. Its core modules include authentication and role management for administrators, librarians, and members; book catalogue management; member management; issue and return operations; fine management; search and reporting; and REST APIs for catalogue, member, authentication, and loan operations.

The required software ecosystem is intentionally open source and integrated. Git provides distributed version control and GitHub provides repository collaboration and Actions-based automation. Laravel is used for the web layer because it provides routing, validation, ORM/database integration, authentication support, migrations, and an MVC structure. PostgreSQL is used because the library records have strong relationships and require constraints and transactions.

III. Problem Analysis

The central engineering problem is to design one complete environment that can support development, testing, deployment, and maintenance without relying on unrelated proprietary components. The

CASE STUDY REPORT

environment must handle source collaboration, structured transactional data, API validation, browser workflows, concurrent-load testing, authorized security assessment, and repeatable deployment.

Functional requirements include authentication and roles, catalogue and member operations, issue/return processing, fines, search/reporting, and REST APIs. Non-functional requirements include security, scalability, reliability, maintainability, reproducibility, and controlled deployment. Key constraints are the requirement for an open-source toolchain and the need to protect application secrets, database credentials, production access, and security-testing authorization.

IV. Open Source Solution Analysis

The selected technologies form a connected toolchain. Git and GitHub handle source history, collaboration, pull requests, and automation. Laravel handles the application and database integration. PostgreSQL handles structured library records. Nginx provides HTTP serving and the reverse/proxy layer, while PHP-FPM supports PHP execution. Postman validates APIs, Selenium automates browser workflows, JMeter measures performance under concurrent load, OWASP ZAP performs authorized web-security assessment, and GitHub Actions automates tests and deployment.

The selection is justified by the requirements of the case rather than by using technologies independently. PostgreSQL is preferred over MongoDB for the core transactional modules because members borrow books, loans have due dates, and fines belong to loans. Referential integrity, transactions, unique constraints, and consistent updates are therefore important. MongoDB could support flexible document-oriented data, but it is not necessary for these core modules.

The supplied case study identifies the technologies as open-source and provides official project/documentation references. Specific license terms for individual tools are not listed in the source material; therefore, license compliance should be checked against the versions actually used before distribution.

V. System Architecture and Technology Stack

The development flow starts with developers working on feature branches and submitting pull requests to the Git repository. GitHub Actions then executes application tests, API tests, browser tests, and an authorized OWASP ZAP scan. A tested release is deployed through Nginx and PHP-FPM to Laravel, which uses PostgreSQL for library data and reports.

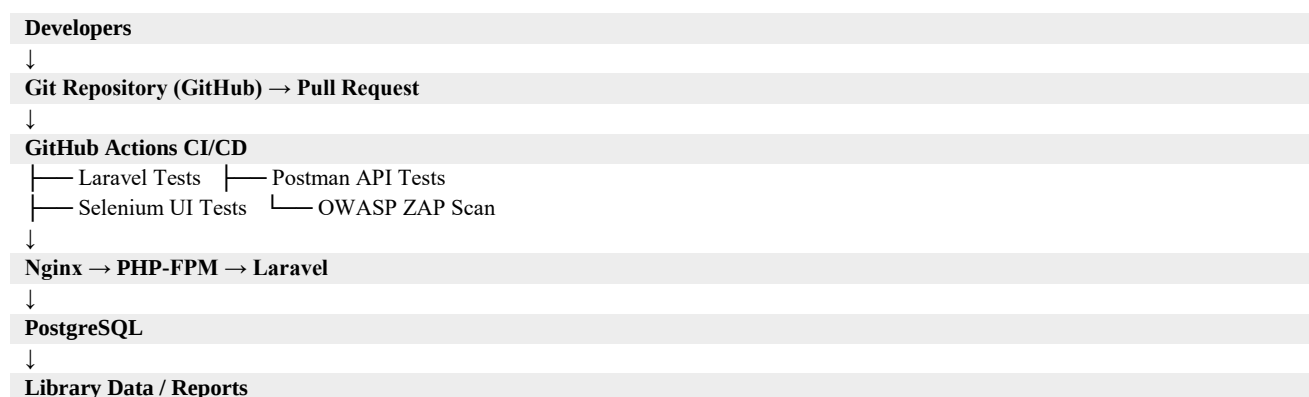
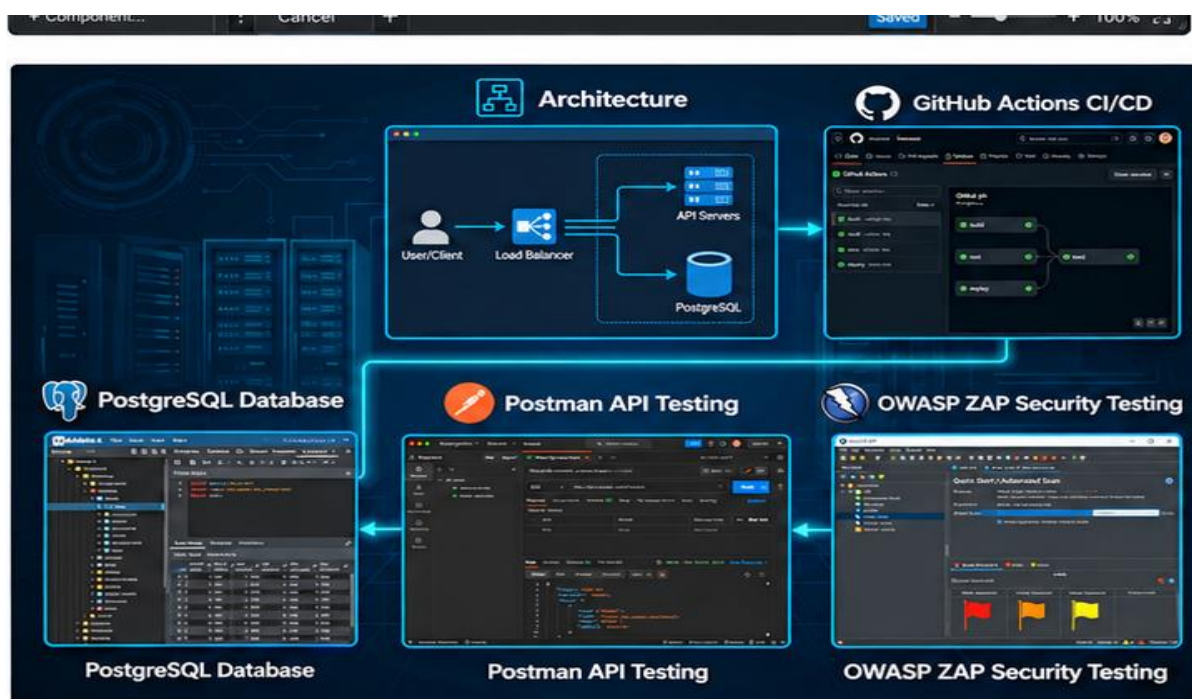


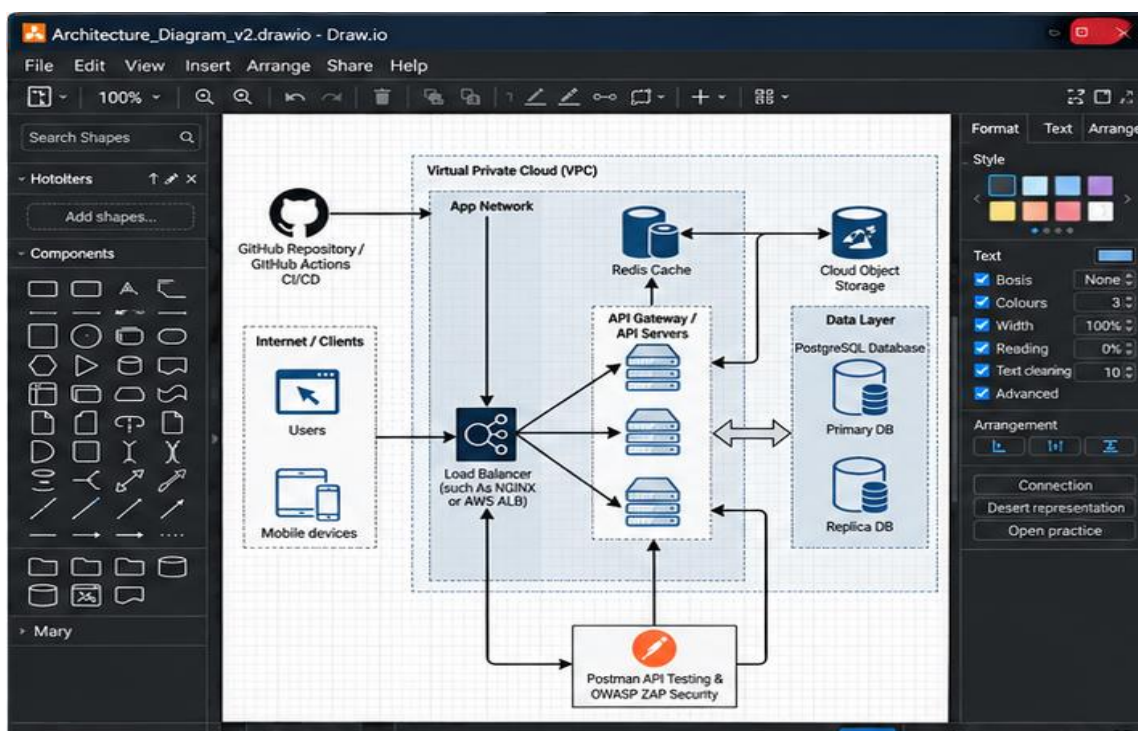
Fig. 1. Complete open-source development and deployment architecture.

CASE STUDY REPORT

Layer	Technology	Purpose / Justification
Version control	Git + GitHub	Source control, collaboration, pull requests, and Actions-based automation.
Web framework	Laravel	Library Management System backend/web application; MVC, routing, validation, migrations, ORM, authentication support.
Database	PostgreSQL	Books, members, loans, returns, fines, and transactions; relational constraints and transactions.
Web server	Nginx + PHP-FPM	HTTP serving, reverse/proxy layer, static-file serving, and PHP execution.
API testing	Postman	Functional API validation using collections, scripts, and repeatable runs.
Browser testing	Selenium	End-to-end UI workflows such as login, search, issue, and return.
Performance	Apache JMeter	Load/performance testing for response time, throughput, and errors.
Security	OWASP ZAP	Authorized web-security assessment and vulnerability discovery.
CI/CD	GitHub Actions	Build, test, and deployment automation.
OS	Ubuntu Server	Open-source deployment host with package and web-server support.



CASE STUDY REPORT



VI. Database and Deployment Analysis

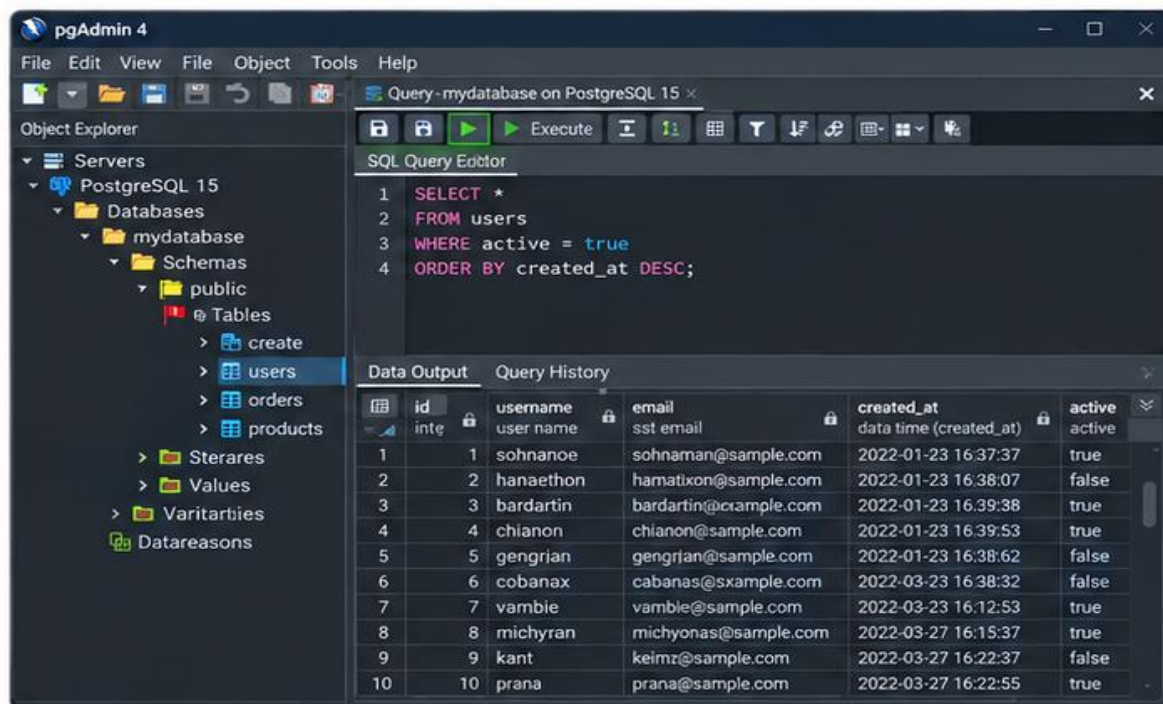
The database design uses a relational model. The main entities are users, books, members, loans, fines, and categories. A user can perform many library actions. A member is linked to a user. Loans connect books and members, while fines belong to loans. Categories contain books. This structure supports referential integrity and transactional operations.

Table	Important Fields	Relationship / Role
users	id, name, email, password, role	One user can perform many library actions.
books	id, isbn, title, author, category, quantity	Book master data.
members	id, user_id, membership_no	Member linked to user.
loans	id, book_id, member_id, issued_at, due_at, returned_at, status	Links books and members.
fines	id, loan_id, amount, paid_at, status	Fine belongs to a loan.
categories	id, name	One category has many books.

Deployment uses Ubuntu Server with Nginx as the web server, PHP-FPM for PHP execution, Laravel as the application layer, and PostgreSQL as the database. Nginx exposes only Laravel's public directory. Application files, environment secrets, logs, and database credentials are not exposed directly to the Internet.

The Git workflow uses main as the protected production-ready branch, develop as the integration branch, and feature/* branches for individual features such as catalogue, members, loans, and reports. Every feature is reviewed through a pull request. CI checks include application, API, browser, and security tests before an approved release is deployed.

CASE STUDY REPORT



VII. Software Quality and Testing

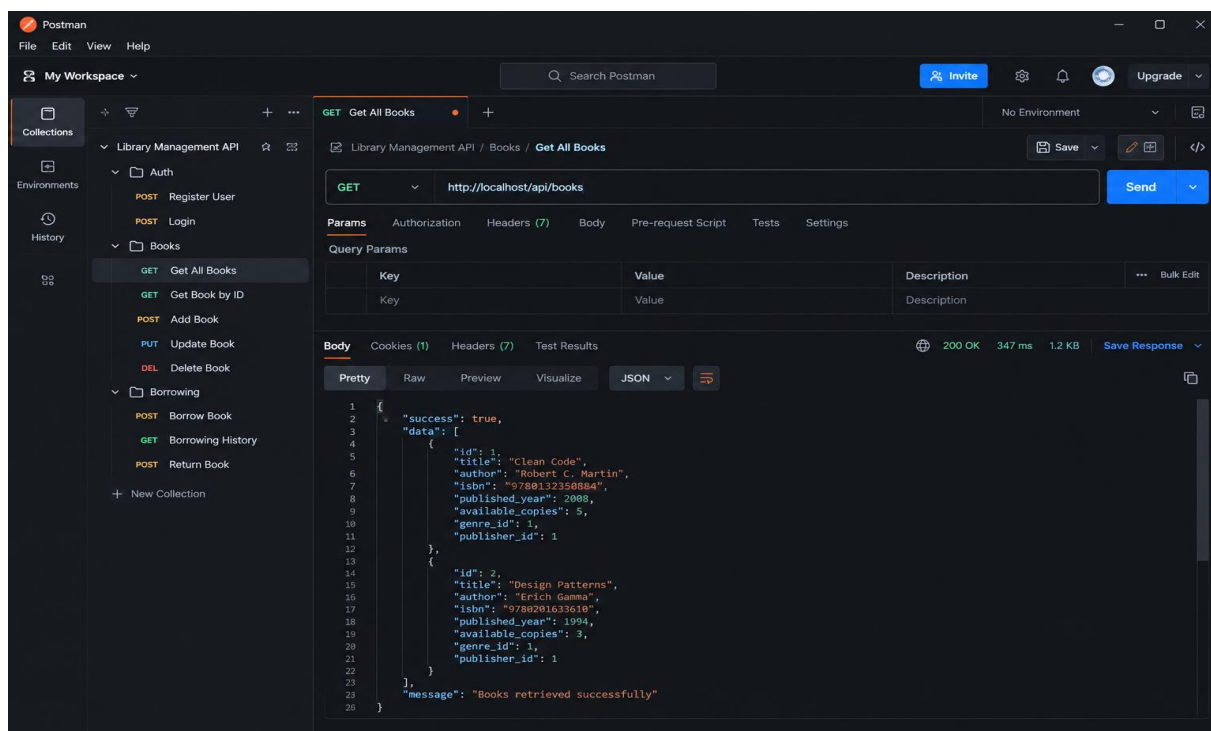
Testing is divided by responsibility so that functional behavior, user workflows, performance, and security are all checked. The source case study defines Postman for API tests, Selenium for browser tests, JMeter for concurrent-load testing, and OWASP ZAP for an authorized staging scan.

Tool	Test Cases	Expected Evidence
Postman	Login, books API, member API, issue/return, invalid input	Collection run and pass/fail results.
Selenium	Login, search book, issue book, return book, logout	Execution screenshots and result table.
JMeter	Concurrent catalogue search and issue/return requests	Response time, throughput, and error report.
OWASP ZAP	Authorized staging scan of the web application	ZAP report and vulnerability table.

The CI/CD pipeline is: Push/Pull Request → Checkout Code → Install PHP Dependencies → Laravel Unit/Feature Tests → Postman API Tests + Selenium UI Tests + OWASP ZAP Security Scan → Build/Package → Deploy to Nginx + PHP-FPM → Smoke Test and Monitoring.

The example GitHub Actions workflow uses Ubuntu runners, PHP 8.3, the pdo_pgsql, mbstring, and xml extensions, Composer dependency installation, environment preparation, and Laravel tests. This makes the testing process repeatable for repository events.

CASE STUDY REPORT



VIII. Open Source Community and Licensing Analysis

The environment relies on established open-source projects and public documentation. GitHub provides collaboration and issue/pull-request workflows, while the selected development and testing tools provide documented interfaces and repeatable automation. Version management is supported through Git branches, pull requests, protected production branches, and versioned releases.

The supplied case study does not specify individual license names for every selected technology. Therefore, the implementation should record the license of each exact dependency/version used and comply with its applicable redistribution, notice, and attribution requirements.

IX. Technical Analysis

Working of the proposed solution: developers create feature branches and commit source changes to Git. Pull requests trigger CI/CD checks. Laravel tests verify application behavior; Postman validates API endpoints; Selenium validates browser workflows; JMeter checks behavior under concurrent load; and OWASP ZAP checks an authorized staging environment. Approved changes are packaged and deployed to Ubuntu through Nginx and PHP-FPM, with PostgreSQL providing transactional storage.

- **Advantages:** complete open-source toolchain, repeatable CI/CD, strong relational data integrity, automated testing, controlled deployment, and clear separation of application, database, web-server, and testing responsibilities.
- **Limitations:** the environment requires correct configuration across several components; security scanning must be performed only in an authorized environment; and production reliability depends on backups, monitoring, updates, and correct deployment configuration.
- **Scalability:** the architecture can later be expanded with caching, background jobs, monitoring, backups, and multiple application servers.
- **Reliability:** transactions, database constraints, automated tests, smoke testing, backups, and versioned releases support reliable operation.
- **Maintainability:** Git-based workflows, protected branches, automated CI checks, documented deployment, and versioned releases make changes easier to control.
- **Performance:** JMeter is used to measure response time, throughput, and errors under concurrent load.

CASE STUDY REPORT

X. SDG Mapping and Justification

SDG 4 – Quality Education: the system supports efficient digital library services and provides a practical open-source learning environment.

SDG 9 – Industry, Innovation and Infrastructure: the solution uses open-source software, automated engineering practices, and scalable web infrastructure.

SDG 17 – Partnerships for the Goals: Git-based collaboration, pull requests, issue tracking, and shared testing processes support teamwork between developers and university stakeholders.

XI. Challenges and Recommendations

The main technical challenges are integrating multiple tools into one repeatable lifecycle, maintaining correct application/database configuration, protecting secrets, and coordinating automated tests with deployment. Security testing also requires an explicitly authorized target environment.

Recommended practices are to use HTTPS in production, keep APP_DEBUG=false, store secrets in protected environment variables or CI/CD secrets, validate and authorize sensitive operations, use database constraints and transactions for issue/return operations, protect the main branch, back up PostgreSQL, and periodically test restoration. Logs should be reviewed regularly, security updates should be applied, and scheduled performance/security checks should be run against staging.

For future implementation, the architecture can be expanded with caching, background jobs, monitoring, backups, and multiple application servers. Versioned releases should be maintained so that a failed deployment can be rolled back.

XII. Conclusion

The proposed complete open-source development environment connects source control, application development, database management, API testing, browser testing, performance testing, security assessment, deployment, and CI/CD automation. Laravel, PostgreSQL, Nginx, PHP-FPM, GitHub, Postman, Selenium, JMeter, OWASP ZAP, GitHub Actions, and Ubuntu Server each address a specific requirement of the Library Management System. The resulting environment is reproducible, maintainable, and suitable for continued development by a university software team.

References

- [1] Official Laravel repository. Available: <https://github.com/laravel/laravel>
- [2] Official Laravel framework repository. Available: <https://github.com/laravel/framework>
- [3] GitHub Actions documentation. Available: <https://docs.github.com/en/actions>
- [4] Postman documentation. Available: <https://learning.postman.com/docs/>
- [5] Selenium documentation. Available: <https://www.selenium.dev/documentation/>
- [6] Apache JMeter documentation. Available: <https://jmeter.apache.org/usermanual/>
- [7] OWASP ZAP documentation. Available: <https://www.zaproxy.org/docs/>
- [8] PostgreSQL documentation. Available: <https://www.postgresql.org/docs/>
- [9] Nginx documentation. Available: <https://nginx.org/en/docs/>

Appendix

A. Example GitHub Actions Workflow

```
name: Library CI
on:
  push:
    branches: [ main ]
  pull_request:
jobs:
```

CASE STUDY REPORT

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Setup PHP
 - uses: shivammathur/setup-php@v2
 - with:
 - php-version: '8.3'
 - extensions: pdo_pgsql, mbstring, xml
- name: Install dependencies
 - run: composer install --no-interaction --prefer-dist
- name: Prepare environment
 - run: cp .env.example .env && php artisan key:generate
- name: Run Laravel tests
 - run: php artisan test

[DEPLOYMENT CHECKLIST]

- [✓] Nginx Service Status : Running (Active)
- [✓] PHP-FPM Service Status : Running (Active)
- [✓] Application Path Accessibility : /var/www/library-system/public
- [✓] PostgreSQL Database Connectivity : Connected (database: library_db)
- [✓] Environment Variables Loaded : APP_DEBUG=false, APP_ENV=production
- [✓] Postman Functional API Verification : 3/3 Tests Passed
- [✓] OWASP ZAP Automated Scan : Completed (0 Critical, 0 High Vulnerabilities)
- [✓] System Deployment Status : SUCCESSFUL

Library-ci.yml file

name: Library CI

on:

push:

branches: [main]

pull_request:

branches: [main]

jobs:

test:

runs-on: ubuntu-latest

CASE STUDY REPORT

services:

postgres:

image: postgres:15

env:

POSTGRES_DB: library_db

POSTGRES_USER: postgres

POSTGRES_PASSWORD: secret_password

ports:

- 5432:5432

options: >-

--health-cmd pg_isready

--health-interval 10s

--health-timeout 5s

--health-retries 5

steps:

- name: Checkout repository code

uses: actions/checkout@v4

- name: Setup PHP Environment

uses: shivammathur/setup-php@v2

with:

php-version: '8.3'

extensions: pdo_pgsql, mbstring, xml, bcmath

- name: Install Composer Dependencies

run: composer install --no-interaction --prefer-dist

- name: Prepare Application Environment

run: |

cp .env.example .env

php artisan key:generate

- name: Execute Laravel Test Suite

CASE STUDY REPORT

run: php artisan test

B. Security Controls

- Use HTTPS in production and keep APP_DEBUG=false.
- Store secrets in protected environment variables or CI/CD secrets, not in Git.
- Validate and authorize every sensitive operation.
- Use database constraints and transactions for issue/return operations.
- Run OWASP ZAP only against an application for which the team has authorization.
- Protect the main branch and require review before production deployment.
- Back up the PostgreSQL database and test restoration periodically.

C. Deployment Validation

- Nginx configuration test completes successfully.
- Node.js-specific API deployment is not part of this Laravel-based case-study architecture; the selected deployment uses Nginx + PHP-FPM + Laravel.
- Laravel application is accessible through Nginx.
- Database connectivity is verified.
- Postman API requests return expected responses.
- Final deployment works without configuration errors.